

BABA LAGIN

API Flow Diagrams (UI -> DB)

End-to-end request flow for each API across all layers,
with the actual source file names

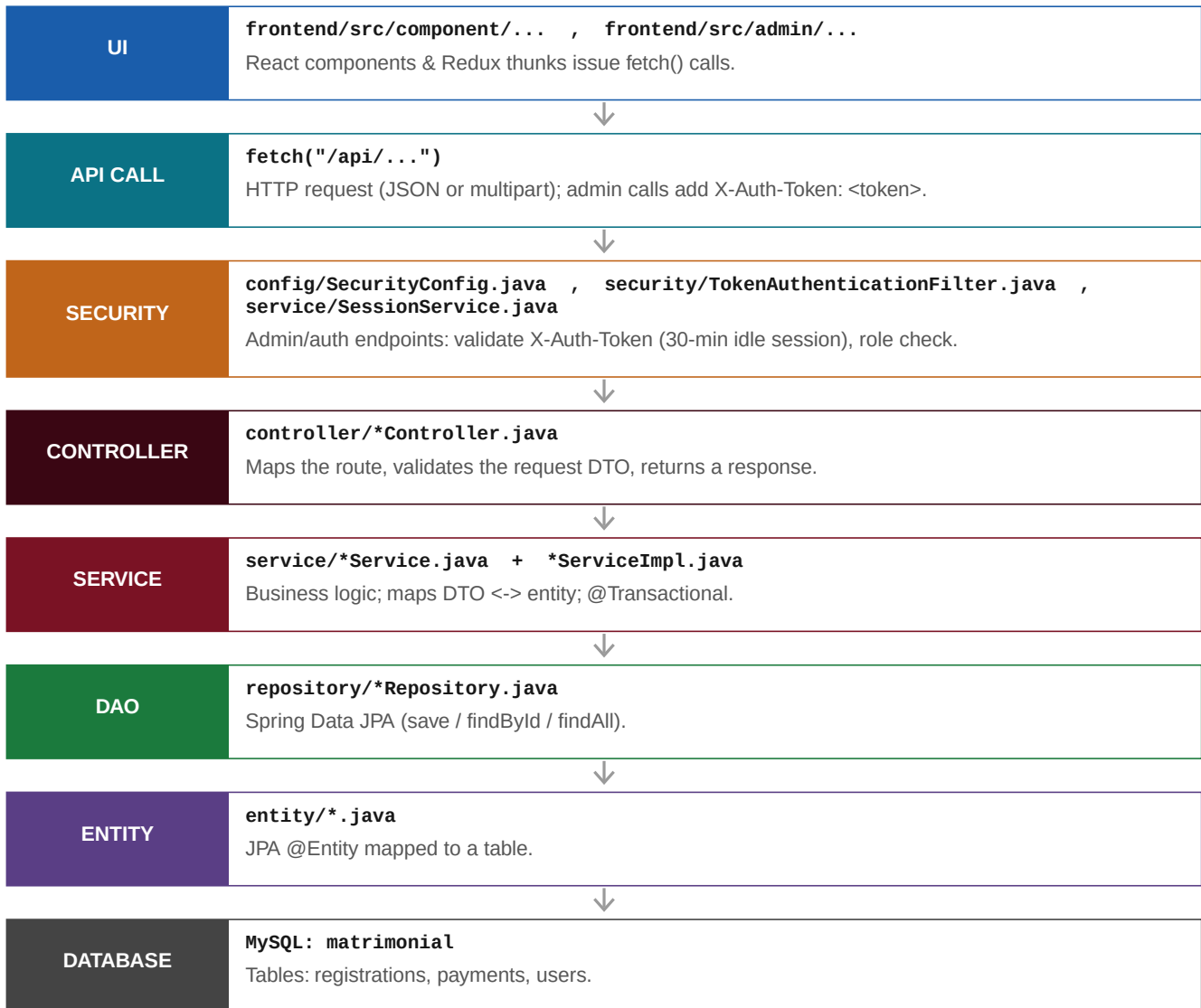
Version	1.0
Date	June 27, 2026
Application	loginapp (Spring Boot 2.7.18 + React)
Layers	UI -> API call -> [Security] -> Controller -> Service -> DAO -> Entity -> DB

1. Architecture & Layers

Every request flows through the same layered stack. The React SPA (built into the Spring Boot static resources) calls a REST endpoint; Spring Security checks access (for admin/auth endpoints); a Controller receives it, delegates to a Service, which uses a Spring Data Repository (DAO) to read/write JPA Entities mapped to MySQL tables. The diagrams that follow show this chain per API with the real file names.

Source locations

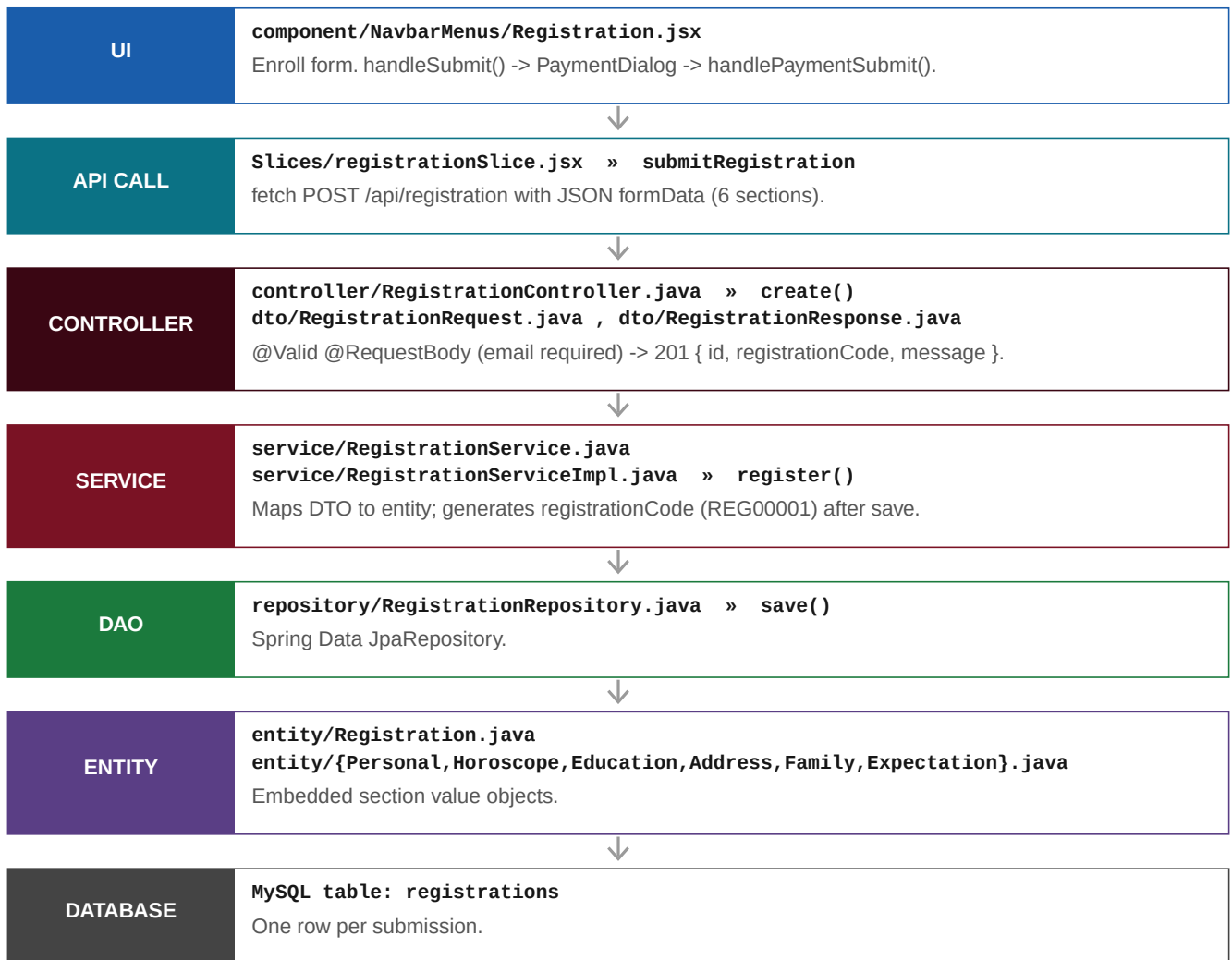
Frontend: frontend/src/ | Backend: src/main/java/com/example/login/ | DB config: src/main/resources/application.properties



2. Registration APIs

POST /api/registration

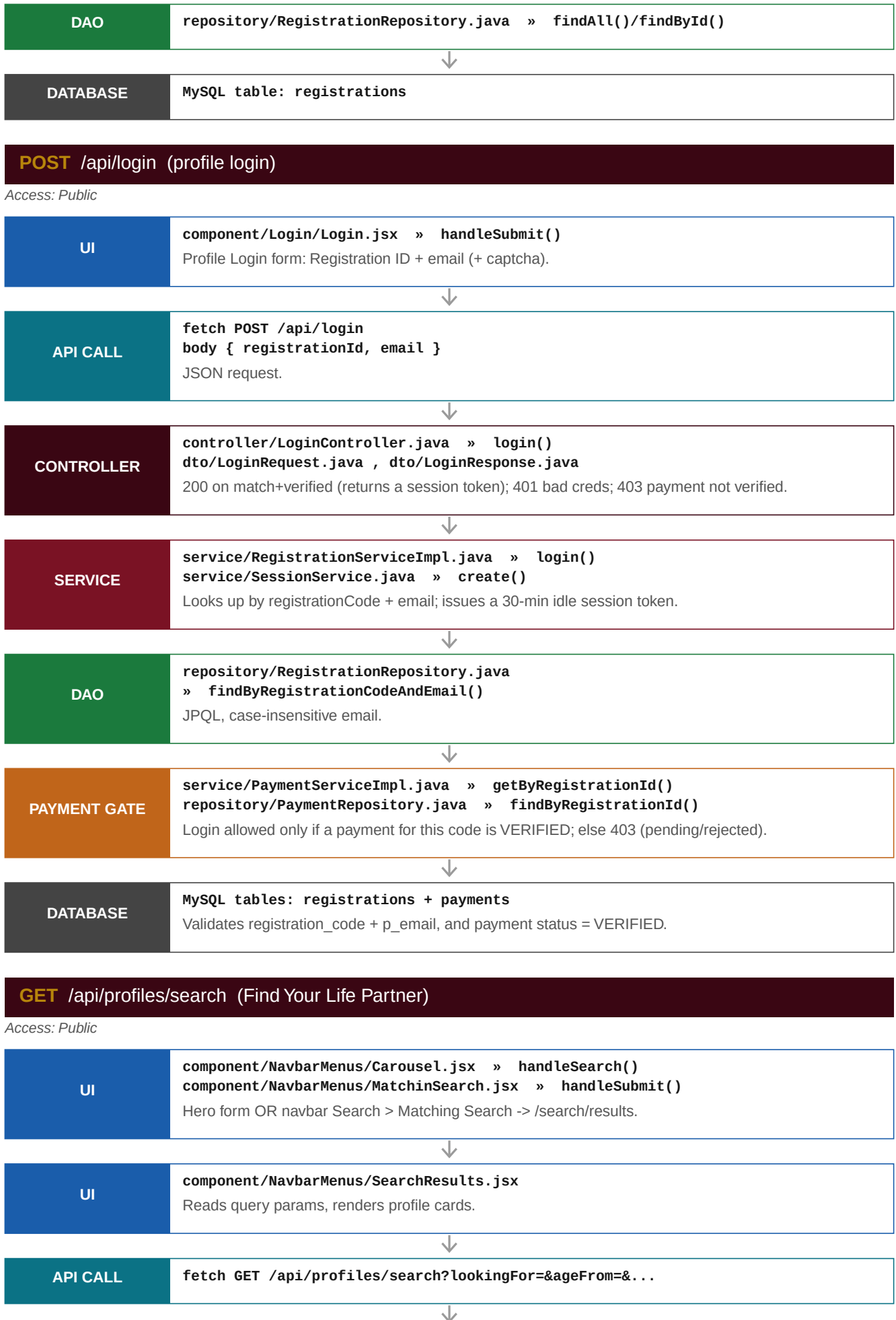
Access: Public

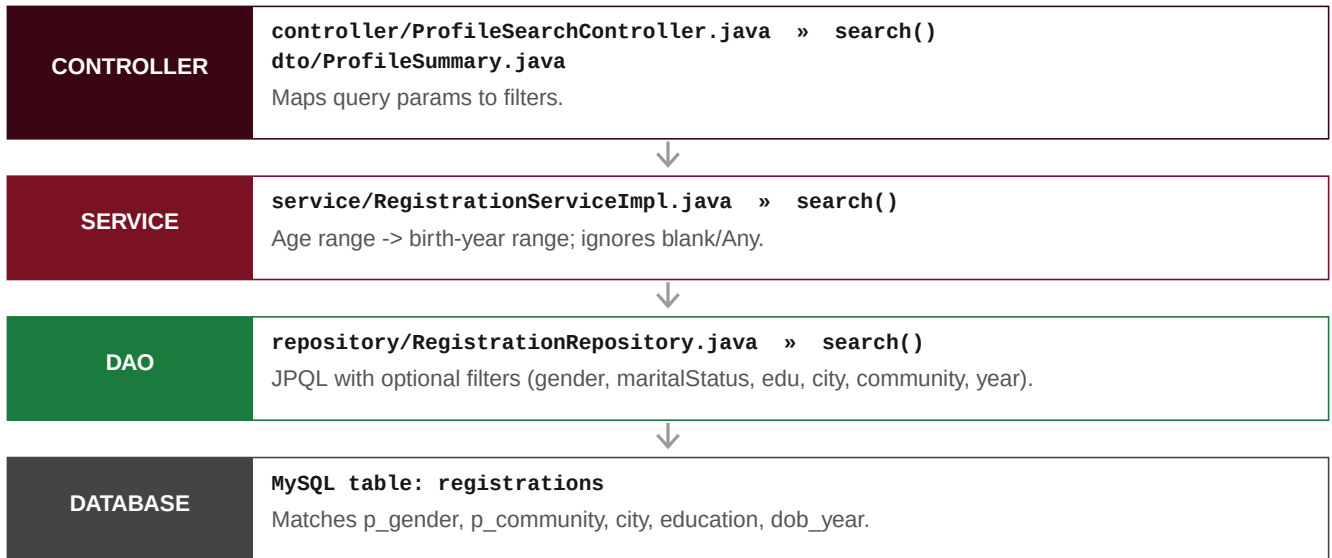


GET /api/registration (and /{id})

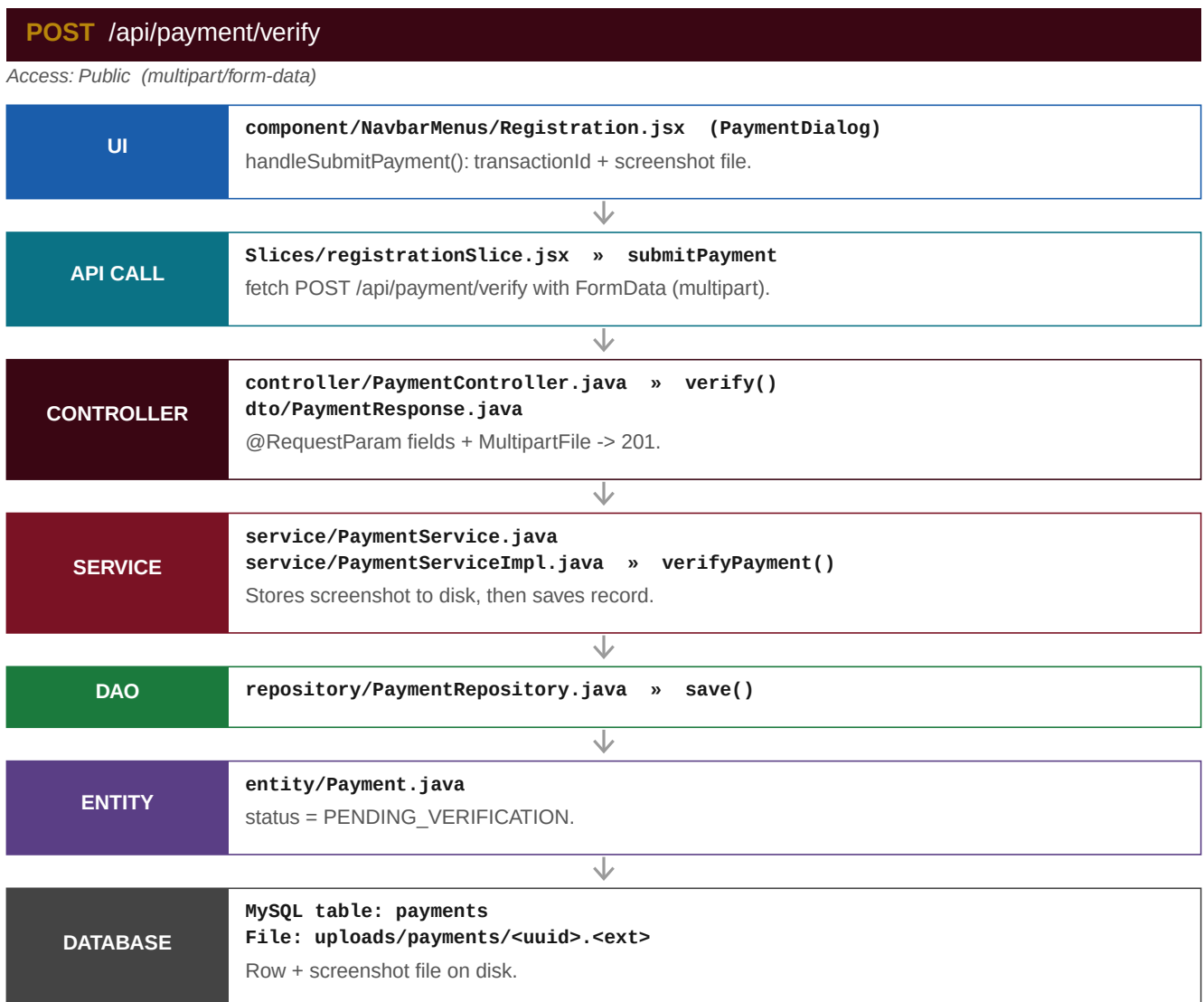
Access: Admin (X-Auth-Token)







3. Payment APIs



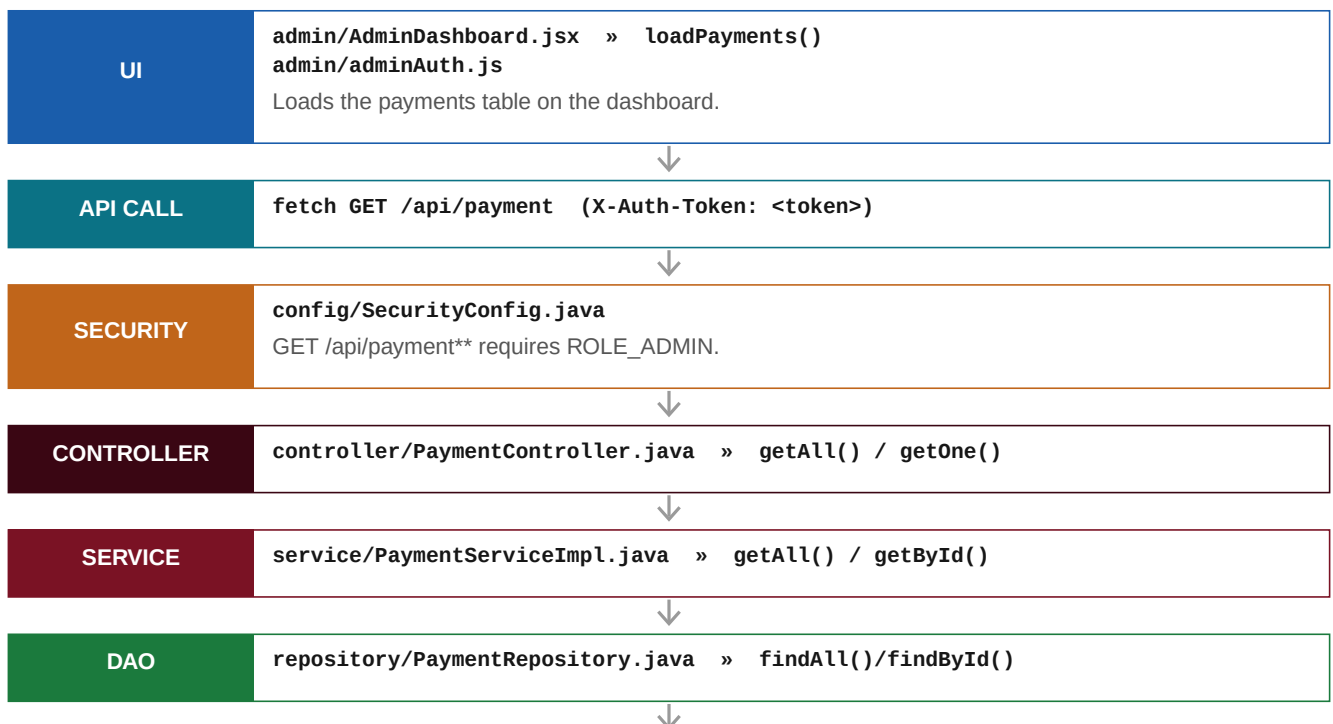
PUT /api/payment/{id}/status

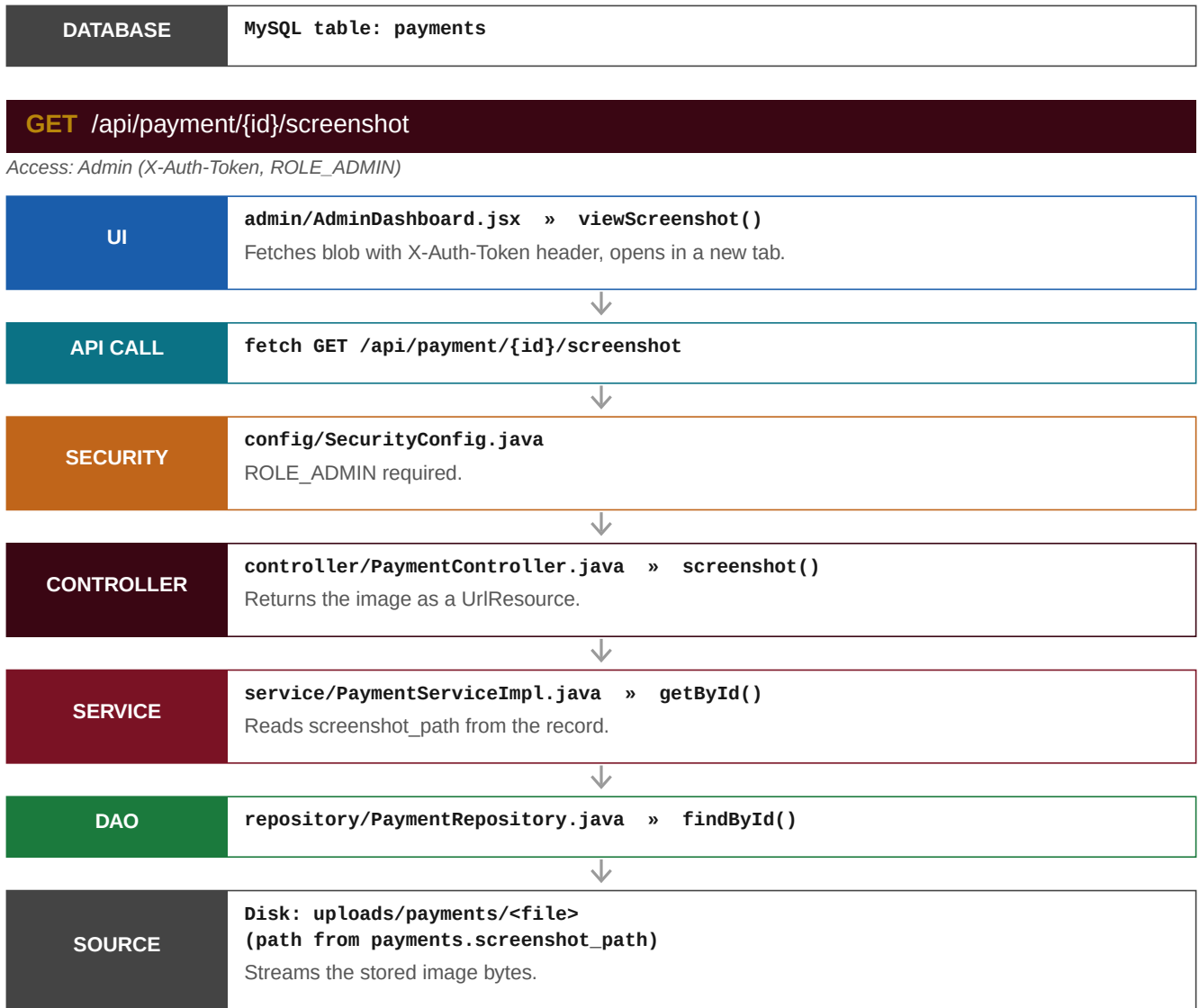
Access: Admin (X-Auth-Token, ROLE_ADMIN)



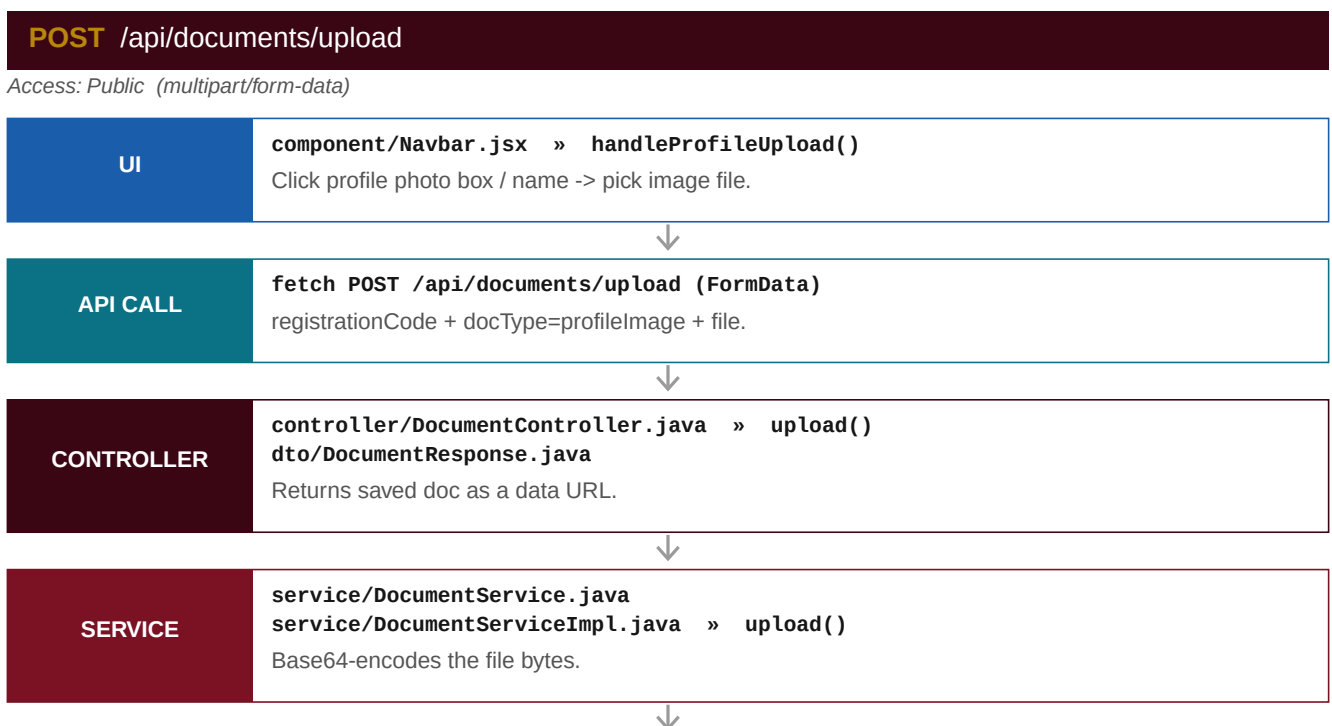
GET /api/payment (and /{id})

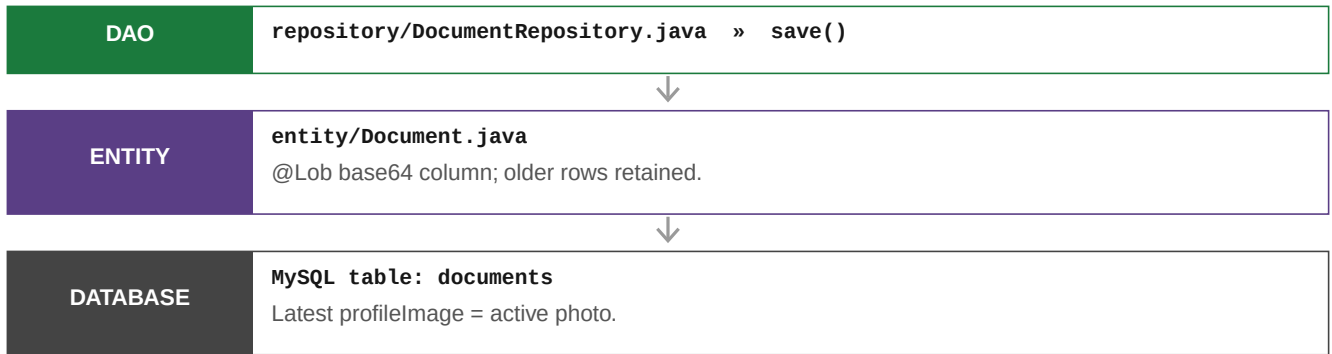
Access: Admin (X-Auth-Token, ROLE_ADMIN)





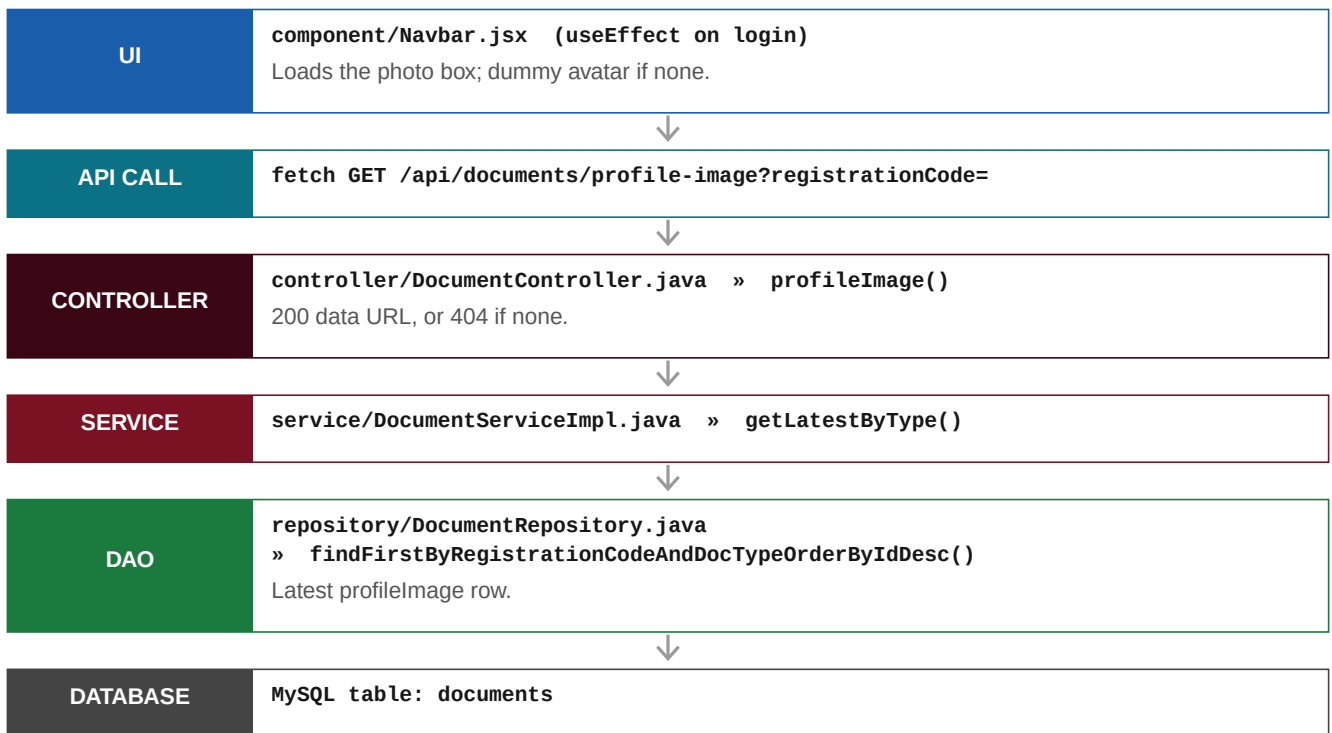
4. Documents (Profile Photo) API





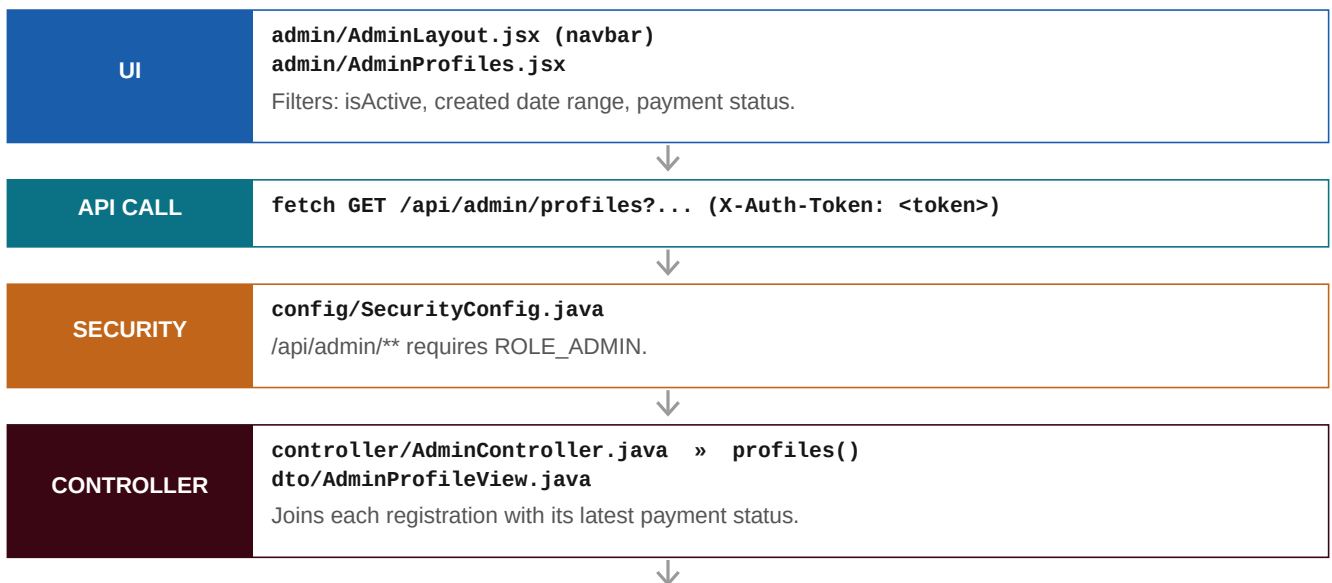
GET /api/documents/profile-image

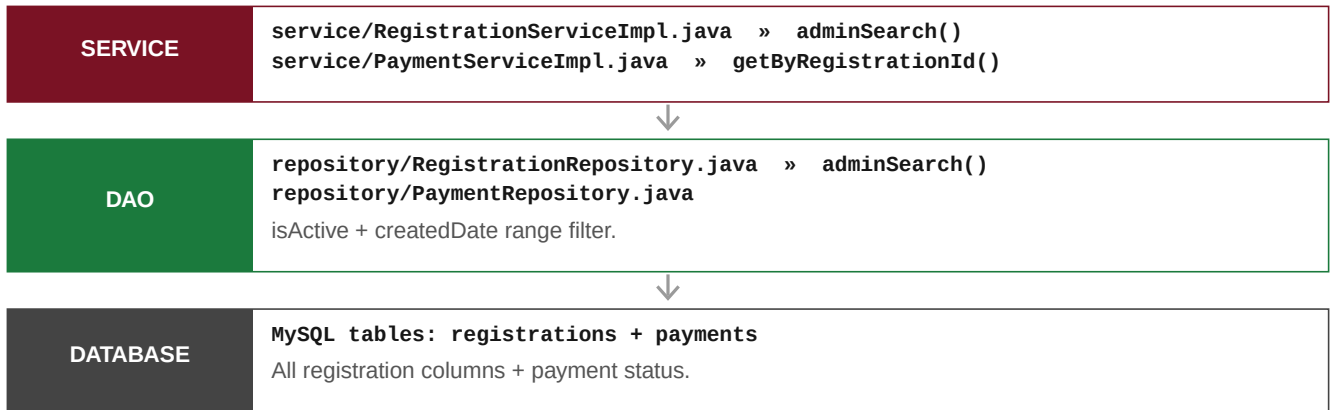
Access: Public



GET /api/admin/profiles (Admin > Profiles)

Access: Admin (X-Auth-Token, ROLE_ADMIN)

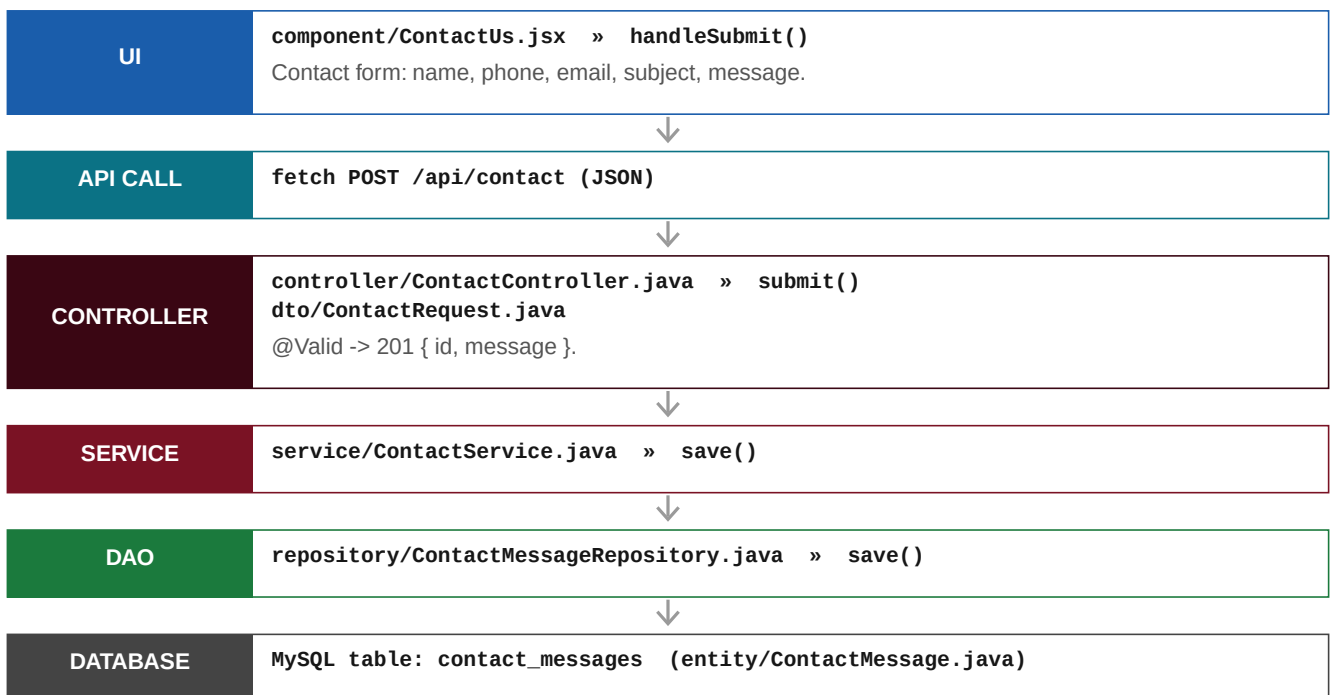




5. Contact Us API

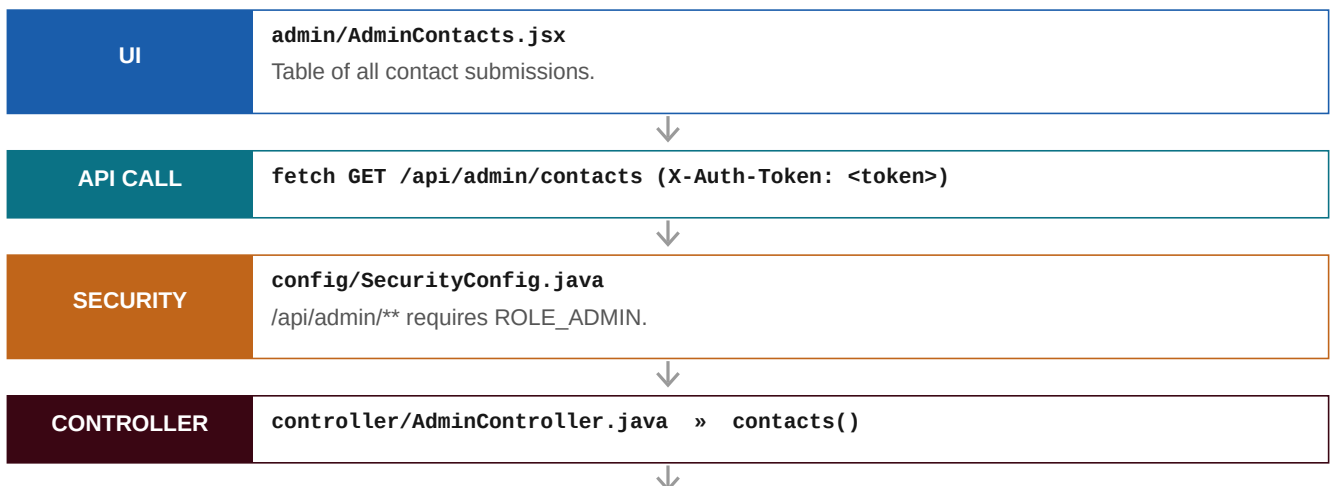
POST /api/contact (public submit)

Access: Public



GET /api/admin/contacts (Admin > Contact Messages)

Access: Admin (X-Auth-Token, ROLE_ADMIN)

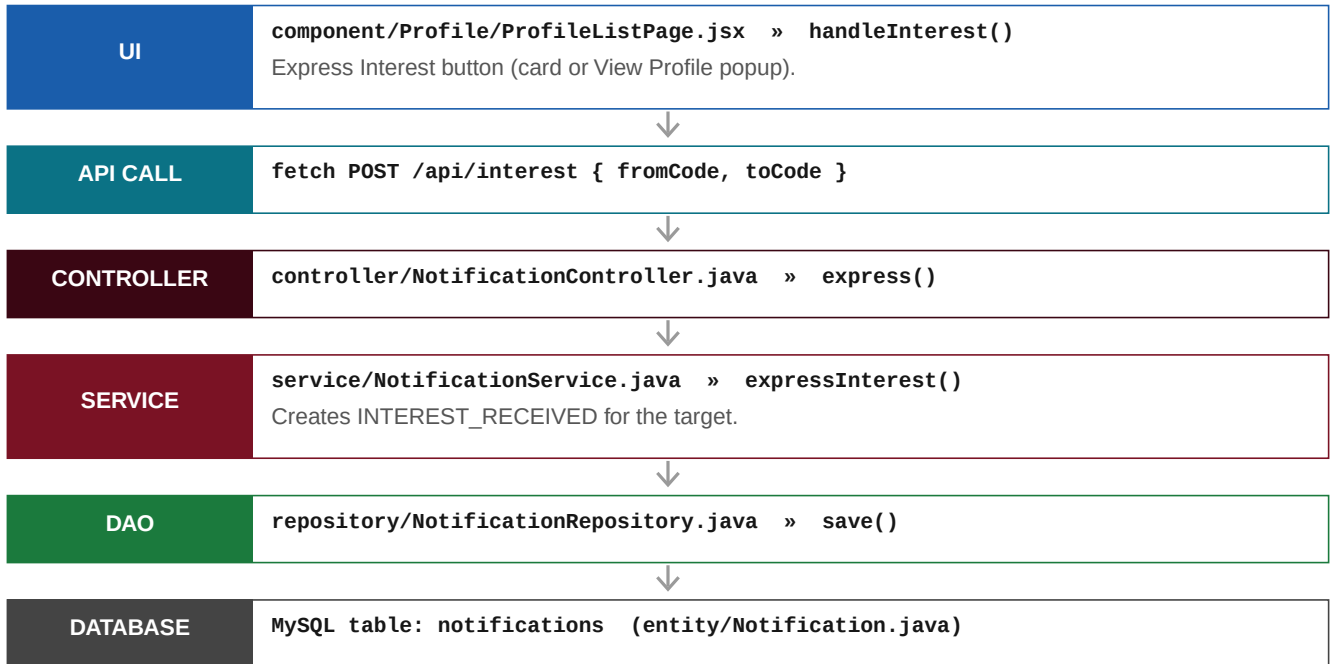




6. Notifications & Interest API

POST /api/interest (express interest)

Access: Logged-in profile user



GET /api/notifications (navbar bell)

Access: Logged-in profile user



DATABASE**MySQL table: notifications**

Also created on admin payment status changes.

7. Authentication API

POST /api/admin/login (admin login)

Access: Public

UI**admin/AdminLogin.jsx » handleSubmit()**

Username + password; requires ROLE_ADMIN to proceed.

**API CALL****fetch POST /api/admin/login**
body { username, password }

On 200, stores token + instanceId via admin/adminAuth.js (saveAuth).

**CONTROLLER****controller/AdminAuthController.java » login()**

200 { token, username, roles, instanceId }; 401 bad creds; 403 not admin.

**SECURITY****repository/AppUserRepository.java**
BCrypt PasswordEncoder

Loads user from DB; verifies password; checks role ADMIN.

**SERVICE****service/SessionService.java » create()**

Issues an in-memory session token with a 30-min idle timeout.

**DATABASE****MySQL table: users (entity/AppUser.java)**
Seeded by config/DataInitializer.java

Admin account read during authentication.

GET/POST /api/session/validate & /api/session/heartbeat

Access: Token (X-Auth-Token)

UI**component/SessionValidator.jsx (profile)**
admin/RequireAdmin.jsx (admin)

Polls validate (every 60s) and sends heartbeat on user activity; logs out on 401.

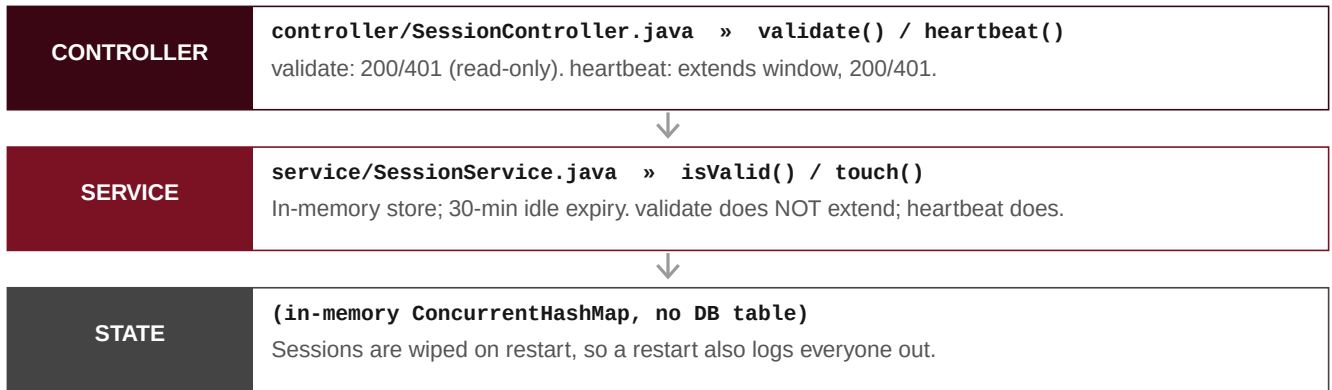
**API CALL****fetch GET /api/session/validate (X-Auth-Token: <token>)**
fetch POST /api/session/heartbeat (X-Auth-Token: <token>)

No body; token in header.

**SECURITY****security/TokenAuthenticationFilter.java**

Reads X-Auth-Token; authenticates if the session is still live.





8. Messaging API

POST / GET /api/messages (chat between accepted connections)

Access: Logged-in profile user

